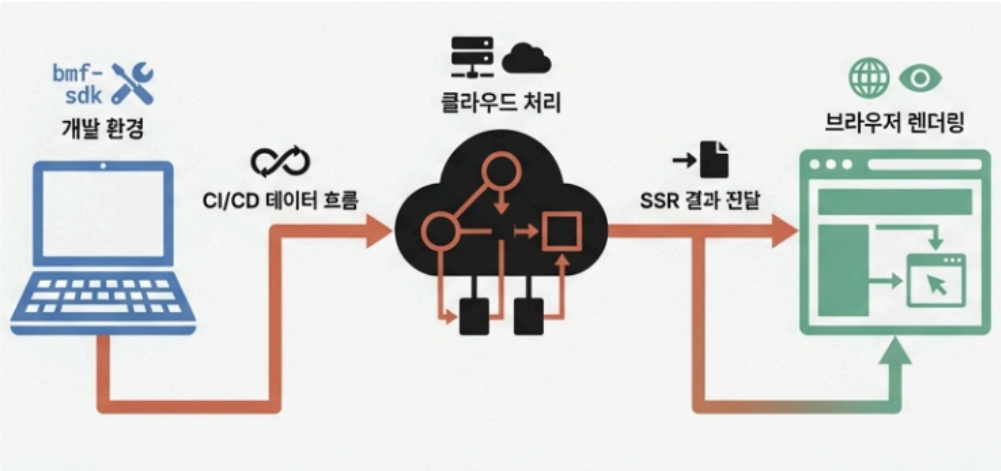
b.stage SDK 워크플로우

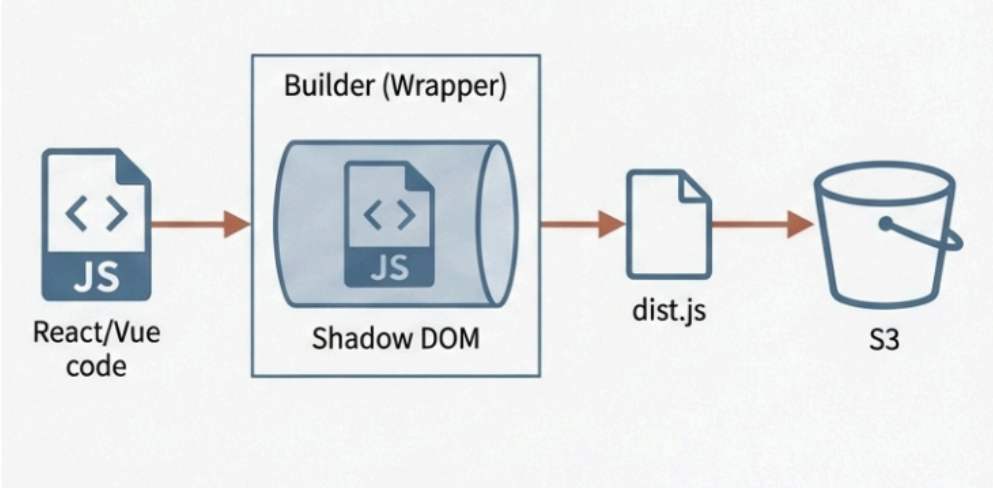
How it works?



b.stage SDK 를 통해 고객사가 자체적인 요구사항을 직접 개발할 수 있도록 SDK 를 제공하고, 이를 통해 산출된 결과물인 **커스텀 페이지** (Web Component 형태)를 BMF 의 인프라에서 통합하여 서비스하는 프론트엔드 아키텍쳐를 소개합니다. 고객사의 개발환경에서부터 BMF 의 FE 애플리케이션 (Next.js), 그리고 최종 사용자 브라우저 렌더링까지 이어지는 시스템의 로직과 흐름을 정리합니다.

참고: [SDK 아키텍쳐 6. 데이터 흐름]

Stage 1. 고객사의 개발 및 빌드환경

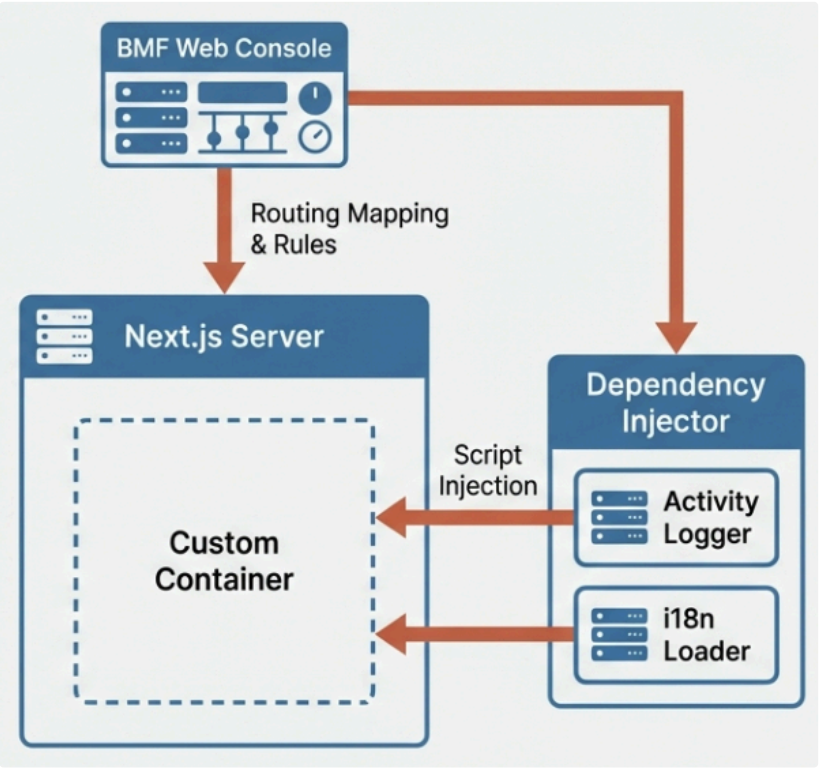


고객사의 개발자는 **커스텀 페이지**를 개발할 때, 필요한 JS 라이브러리로 CSR 환경을 기반으로 하는 비표준 UI 코드를 작성합니다. 이는 SDK 의 Web Component Builder (Web Component Factory) 를 통해 표준 규격으로 변환됩니다. 고객사의 개발자가 작성한 UI 코드를 빌드하게 되면, 표준화된 Web Component 로 변환된 결과물이 `manifest.json` & `template.js` 로 출력되고, 이 결과물은 규칙에 따라 S3 저장소의 특정 위치에 저장됩니다.

핵심 요소:

- **Web Component Factory:** 고객사가 작성한 React/Vue/Vanilla JS 코드를 입력받아 네이티브 Web Component 사양(Custom Elements API)으로 래핑합니다.
  - **Shadow DOM**을 적용하여 고객사 컴포넌트의 CSS 스타일과 DOM 트리가 BMF 메인 서비스와 충돌하지 않도록 완벽히 캡슐화(격리)합니다.
  - 빌드 결과물로 `manifest.json` & `template.js` 파일을 생성합니다.
- **CI/CD 파이프라인 및 S3 Uploader (배포 모듈):**
  - CI 를 통해 빌드된 산출물을 AWS S3 버킷의 정해진 경로(테넌트 ID 및 버전 기반 라우팅)로 자동 업로드 합니다.

Stage 2. 서비스 라우팅 설정 및 렌더링



업로드된 **커스텀 페이지**를 기존 애플리케이션 (Next.js) 구조 내에 서빙하기 위해, 관리자(박애런 혹은 김아리)가 특정 URL 경로(i.e. `/templates/home`)를 어드민 도구를 통해 할당합니다. 할당 과정을 통해 페이지와 컴포넌트 위치(CDN 주소)가 매핑되어 저장됩니다. Next.js 서버가 해당 경로의 요청을 수신하면, Custom Container를 랜더링하며, 컨테이너는 아래 요소들을 포함하여 응답을 반환합니다.

- 고객사 컴포넌트의 위치 주소 (S3) 비동기 스크립트
- 보안, 운영등에 관련된 BMF 애플리케이션용 스크립트 (i.e. Activity Logger, i18n Loader 등)
- 해더 푸터 등 공통 UI 요소 (Optional)

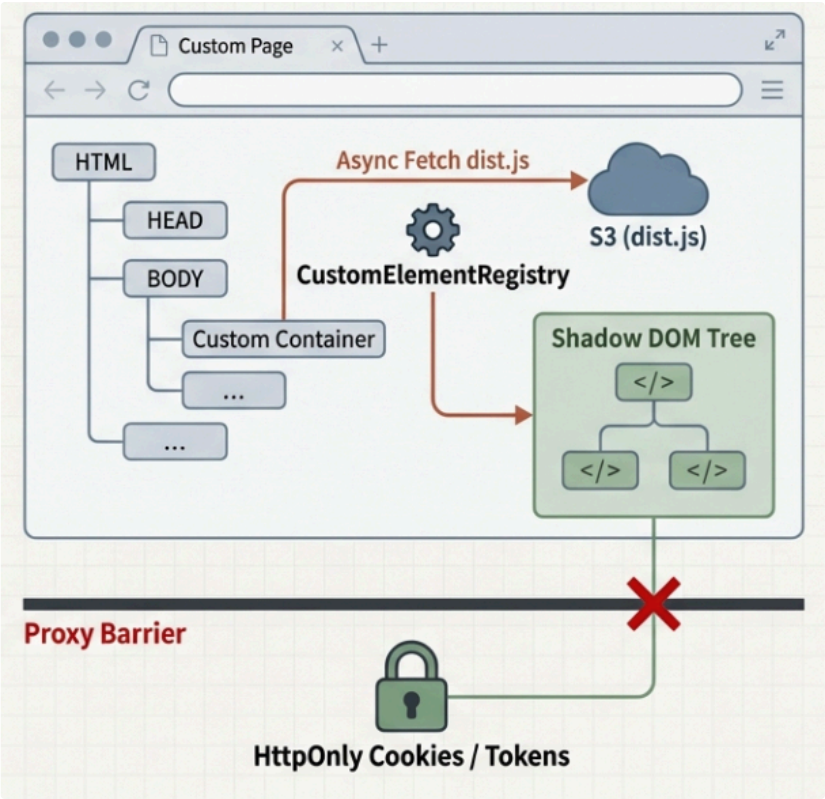
핵심 요소:

- **라우팅 매핑 시스템 (Web Console 혹은 Admin):** BMF 서비스 내의 특정 URL 경로(예: `/template/home`)를 'Custom Page'로 지정하고, 해당 페이지에 렌더링될 S3 Web Component의 버전을 매핑합니다.
- **Custom Container:**
  - Custom Page 진입 시 Next.js 서버에서 렌더링되는 호스트 컨테이너(Wrapper Component)입니다.
  - BMF 에서 관리하는 영역으로, 고객사의 **커스텀 페이지**와 보안, 운영 및 공통 UI 요소 등 필요한 스크립트를 병합하여 서비스합니다.
  - DB에 매핑된 S3 주소를 기반으로 Web Component를 로드하는 비동기 `<script>` 태그를 DOM에 주입합니다.
  - 예시

```
const { loadTemplate } = await import('@partners-bmf/bstage-host');
const tpl = await loadTemplate(manifestUrl);
tpl.mount(containerRef.current);

/**
* 이 때 아래와 같이 custom-component api 를 가지고 mount 됨
* mount 되면서 CLS 이슈 발생할수도 있음: 교체 영역을 미리 Skeleton UI 등으로 할당하기.
* <custom-component ... />
*/
```

Stage 3. 최종 사용자 CSR 과정 및 정보 보안



최종 사용자의 브라우저는 **커스텀 페이지** 초기 로드시 BMF 애플리케이션 (Next.js)의 HTML 응답을 수신합니다. 이때 **Stage 2** 의 Custom Container 를 통해 주입된 고객사 컴포넌트 로딩 스크립트를 실행하여 S3 저장소에서 `dist.js` 를 로드합니다. 로드된 컴포넌트는, Web Component 표준을 따라 Shadow DOM 트리를 전개하여 최종적으로 UI 렌더링을 완료합니다.

주의사항:

고객사의 스크립트는 BMF 에서 온전히 관리책임을 가진 소스가 아니므로, 고객 정보에 접근하는 행위등에 대한 방어책이 필요합니다. 이 소스는 BMF 의 소스와 같은 런타임 환경에 주입되기 때문에, BMF 가 데이터에 접근할 수 있다면 고객사도 접근할 수 있다는 위험성이 있습니다. 따라서, 고객의 Access Token 등 인증 키는 Client-side JS 로는 접근할 수 없는 HttpOnly Cookies 에 보관하여 데이터 유출을 원천차단할 필요가 있습니다.